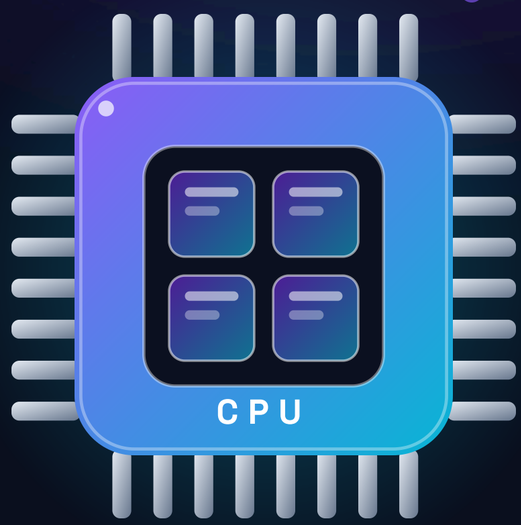
BİL 333 · PROFESSIONAL ENGLISH

How One CPU Runs Many Programs

Context Switching



Emre Kuzal

Computer Engineering · Necmettin Erbakan University

12 November 2026

My talk has three parts

1

What is it?

Processes and the idea of a context switch

2

How does it work?

The steps inside the operating system

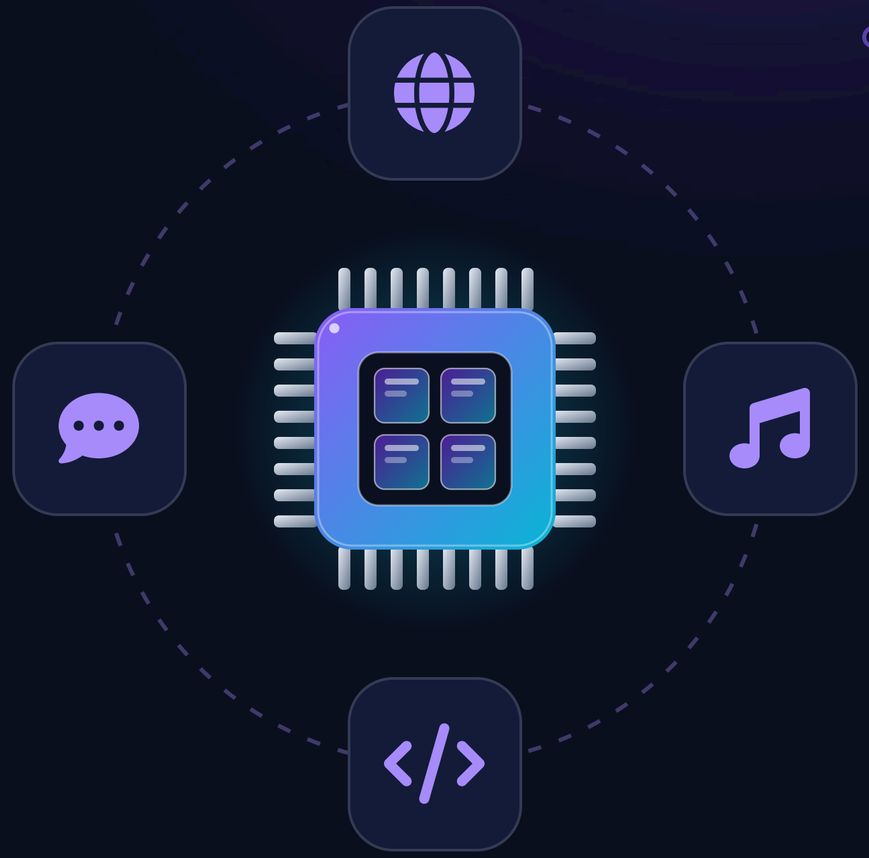
3

What does it cost?

Time, cache and why it matters

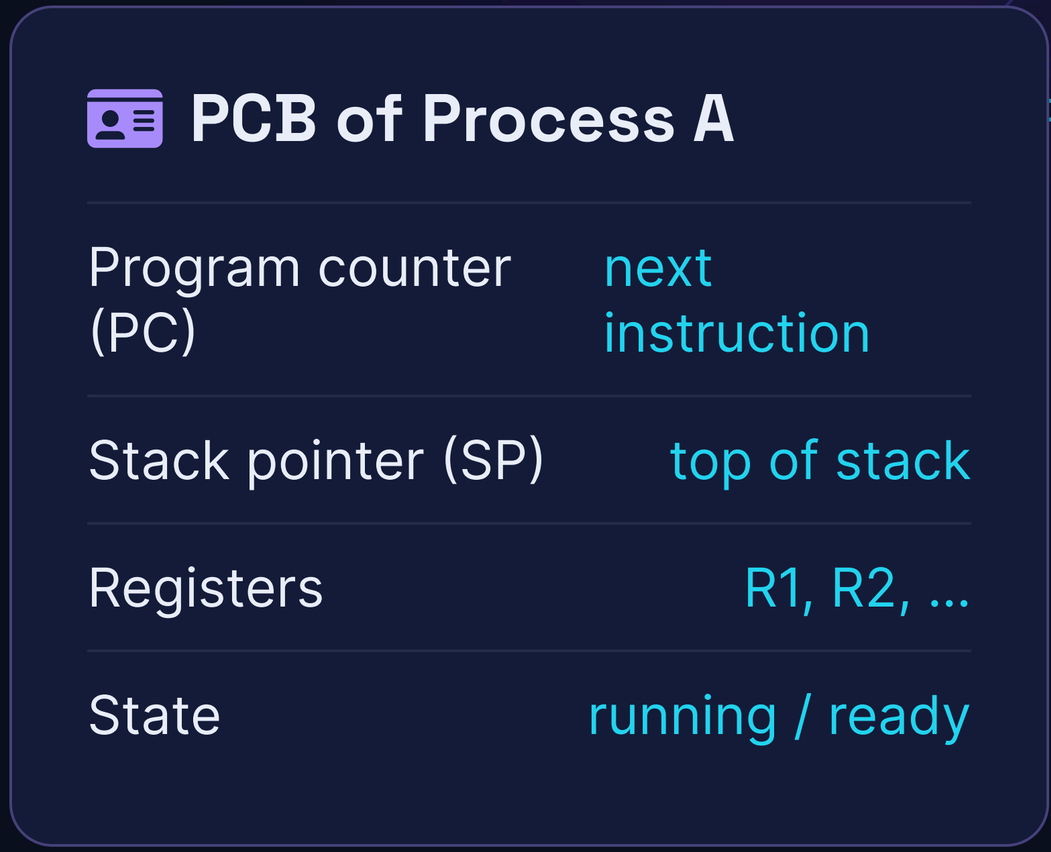
One CPU core, many programs

- One core runs **one program** at a time
- But we run browser, music, chat… together
- The trick: the OS **switches very fast**



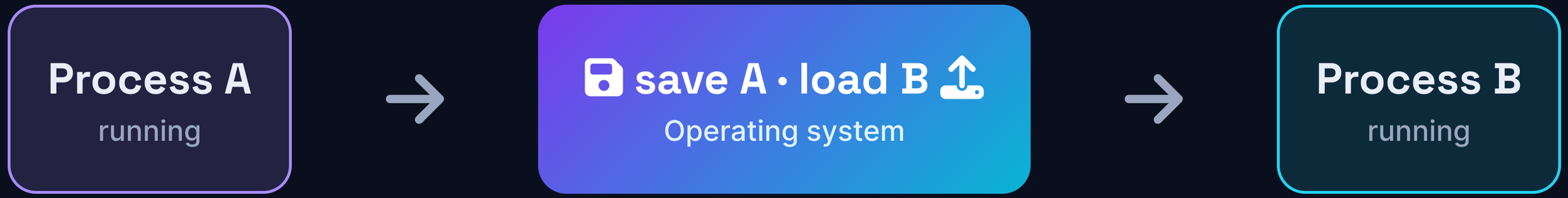
Every program is a process

- Process = a **running** program
- The OS keeps its information in a **PCB**
- PCB = Process Control Block



What is a context switch?

Save the state of one process, load the state of another.



When does it happen?

Timer interrupt
the time slice is over

Waiting for I/O
disk, network, keyboard

Process sleeps
it gives the CPU back

Higher priority
a more important process is ready

Step by step

1

Timer interrupt

The CPU stops A and jumps to the OS

2

Save A

A's registers go into PCB A

3

Choose B

The scheduler picks the next process

4

Load B

B's registers come back, B continues

“Limited Direct Execution” — OSTEP, Chapter 6

A real example: Windows

1

Save

Save the context of the current thread

2

Queue

Put it at the end of its priority queue

3

Find

Find the highest-priority ready queue

4

Run

Load the next thread's context and run it

Microsoft Learn, “Context Switches”

Who chooses the next process?



Linux

CFS → **EEVDF**

since Linux 6.6 (2023)

Goal: a fair share of CPU time



Windows

Priority-based

one queue per priority level

Goal: important threads first

Linux kernel docs (CFS, EEVDF) · Microsoft Learn, “Context Switches”

A switch is fast, but not free

~1.2–2.2 µs

for one switch between Linux threads (Bendersky, 2018)

- Save and load registers
- Jump into the kernel and back
- **Thousands** of switches per second add up

The hidden cost: a cold cache



- The cache is full of **A's data**
- B gets **cache misses** → slower
- Often bigger than the direct cost (Li et al., 2007)

Hardware got faster, but OS work like switching did not keep up — Ousterhout, 1990

See it on Linux

```
$ vmstat 1
 r  b ...   in   cs us sy id
 1  0 ...  812 1534  4  1 95

$ grep ctxt /proc/self/status
voluntary_ctxt_switches:     1
nonvoluntary_ctxt_switches:  0
```

- **cs** = switches per second
- Voluntary: the process waits
- Nonvoluntary: the OS stops it

Linux man-pages: vmstat(8), proc_pid_status(5)

See it on Windows

PowerShell

```
PS> while ($true) {
  (Get-CimInstance Win32_PerfFormattedData_PerfOS_System).ContextSwitchesPersec
  Start-Sleep 1
}
18452
21310
17986
```

Every second
switches per second

Loop
runs until Ctrl + C

Thousands
even on an idle PC

Microsoft Learn: Win32_PerfFormattedData classes

To sum up



One CPU + fast switching = many programs



A switch = save one state, load another



Fast, but not free: fewer switches, faster system

Thank you! Any questions?

Sources

1. Ousterhout, J. (1990). Why Aren't Operating Systems Getting Faster As Fast as Hardware? USENIX — PAPER
2. Li, C., Ding, C. & Shen, K. (2007). Quantifying the Cost of Context Switch. ACM ExpCS — PAPER
3. Arpaci-Dusseau, R. & A. Operating Systems: Three Easy Pieces, Ch. 6: Limited Direct Execution — BOOK
4. Microsoft Learn. Context Switches — learn.microsoft.com — TECH DOC
5. Linux Kernel Documentation. CFS Scheduler & EEVDF Scheduler — docs.kernel.org — TECH DOC
6. Linux man-pages. vmstat(8) & proc_pid_status(5) — man7.org — TECH DOC
7. Microsoft Learn. Win32_PerfFormattedData classes — learn.microsoft.com — TECH DOC
8. Bendersky, E. (2018). Measuring context switching and memory overheads for Linux threads — BLOG